

Build an AI that joins a **live video call**, appears as a person, and conducts a real job interview — grounded in the candidate's actual resume, the actual job description, and their actual GitHub code — then produces a hiring scorecard it can defend with evidence.

1 What you build

Before the call: recruiter uploads a JD + a candidate resume PDF. Your agent parses both, finds the GitHub link, reviews the real repos, and generates a question plan. Recruiter **approves / edits / rejects**. Nothing runs without approval.

The call: candidate joins a link. An AI with a **visible face** and natural voice greets them by name, discloses it is an AI, and interviews them for 8+ minutes. It cites their real projects. Shallow answer → it probes deeper. Strong answer → difficulty rises. Candidate can **interrupt mid-sentence** and it stops talking.

After: scorecard with a score per competency, an **evidence quote from the transcript behind every score**, hire / no-hire + confidence, transcript, recording, PDF report.

Grading standard: a grader watches your recording and asks one question — “would a real candidate believe they were being interviewed?”

2 Workflow

```
PHASE 1 – PREP (LangGraph, offline)
parse_jd      -> competencies, must-haves, seniority
parse_resume  -> roles, claims, skills, links
github_agent  -> GitHub REST: repos, languages, commit
               cadence, READMEs; read real source
gap_analysis  -> resume claim vs JD need vs GH evidence
question_planner -> 12 Qs {competency, source, difficulty}
               >=3 MUST cite the candidate's real code
               [ HITL INTERRUPT – recruiter approves ]

PHASE 2 – LIVE CALL (real-time)
intro -> resume_probe -> jd_fit -> github_deepdive
      -> scenario -> candidate_Qs -> wrap_up
CONDITIONAL EDGES:
  shallow answer -> follow_up (max 2, then move on)
  strong answer  -> raise difficulty
  bluff detected -> verify ("walk me through that commit")
  silence/off-topic -> recovery
  time spent     -> jump to wrap_up
barge-in: candidate cuts in -> agent stops instantly
checkpointer: call drops -> resume at last node

PHASE 3 – SCORING
transcript -> 1-5 + evidence quote + confidence
guardrail  -> score without a quote is REJECTED
outputs    -> scorecard.json, transcript.json, report.pdf
MCP server -> recruiter queries it from Claude Desktop
```

3 Core requirements — all 12 required

#	Requirement	What it means	Proof
1	Live video call	Real-time two-way call. AI has a visible face + natural voice. Minimum 8 minutes continuous.	Raw recording
2	Barge-in	Candidate interrupts mid-sentence → agent stops speaking and listens. No talking over each other.	Timestamp in SUBMISSION.md
3	JD + resume parsing	PDF resume and JD parsed into structured JSON. No manual copy-paste of fields.	output/prep/*.json
4	GitHub grounding	Real GitHub pulled via API. ≥3 questions cite a specific repo, file, or commit . Generic questions do not count.	question_plan.json
5	LangGraph	Typed state, ≥6 nodes , ≥2 conditional edges , SQLite checkpointer so a dropped call resumes.	graph.py + diagram
6	Adaptive follow-up	Shallow answer triggers a deeper probe. Max 2 probes per topic , then move on. Infinite loop = fail.	Transcript
7	HITL gate	Graph genuinely pauses for recruiter approval of the question plan. Approve, edit and reject must all work.	Screenshot + code
8	Custom MCP server	≥5 tools (get_candidate, get_question_plan, save_score, get_scorecard, list_interviews). Must work in Claude Desktop, not only inside your app.	Screenshot of tools in Claude Desktop
9	Guardrails	(a) banned questions blocked before reaching the candidate — age, gender, marital, religion, nationality, health/pregnancy, salary history, politics; (b) no score without a transcript quote .	guardrails/ + test proving both fire
10	Scorecard	Per-competency score, evidence quote, hire / no_hire / borderline, confidence. Schema in \$6.	output/scorecard.json
11	Evals	Run your scorer on the 5 personas (\$9). Must rank Strong > Average > Weak. Report failures honestly.	evals/results.md
12	Prompts as files	Every system prompt its own file, with v1—v2 notes : what the first version got wrong, what you changed.	prompts/

The face is mandatory — the vendor is not. Voice-only does not satisfy #1. But a simple **2D avatar** (portrait image + viseme lip-sync driven by the TTS stream) earns **full marks**. Do not spend your 24 hours chasing a photorealistic avatar.

4 Stack — free tiers only, fallbacks lose no marks

Layer	Recommended (free)	Fallback
Realtime brain	Gemini Live API — native audio in/out, built-in interruption	Groq Whisper STT + Groq LLM + free TTS pipeline
Transport	LiveKit Cloud free tier + LiveKit Agents	Plain WebRTC, or Daily.co free tier
AI face	Simli / Tavus / HeyGen trial credits (LiveKit plugins exist)	2D lip-synced avatar — full marks
Graph	LangGraph + SQLite checkpointer	—
GitHub	REST API + free PAT (5,000 req/hr)	Unauthenticated (60/hr) — cache hard
Resume	pypdf / pdfplumber + LLM structured output	—
MCP	fastmcp (Python) or TS MCP SDK, stdio	—
Report	reportlab / WeasyPrint / HTML→PDF	Markdown report, −1
Deploy	Render / Railway free tier	Localhost (bonus mark only)

Free-tier protection: nobody loses marks because a trial credit expired. Every one of the 12 requirements is fully achievable on the fallback column. **Latency target: <1.2s** from candidate stops speaking to agent starts. Measure it, put the number in ARCHITECTURE.md.

5 Deliverables

#	Item	Spec
1	Code explanation video 1 min	Screen + your own voice . Must show: your graph, one conditional edge, your MCP tools, one guardrail firing. Script it — 60s is short.
2	Live demo video 1 min 30 s	Edited highlights: AI face speaking, a barge-in moment, a GitHub-grounded question, the final scorecard.
3	Raw recording 8 min minimum	Unedited, one take, full interview. Proof the demo is real. No raw recording → demo video not graded.
4	GitHub repo	Public. ≥10 commits spread across the 24h. One squashed commit is penalised.
5	output/ files	Exact paths from \$6. Graded automatically — wrong path = zero for that item.
6	ARCHITECTURE.md	Graph diagram, state object, measured latency, known limitations.
7	prompts/	Prompt files + v1→v2 iteration notes.
8	evals/results.md	5 personas, ranking table, honest failure analysis.
9	MCP proof	Screenshot of your tools live in Claude Desktop.
10	SUBMISSION.md	One page: links, what works, what is broken, barge-in timestamp, JD chosen, latency, avatar used, bonus attempted, candidate consent line .
11	Viva	10 min live, within 48h of the deadline. Pass/fail gate (\$8).

Video rules: your own voice, no AI narration. Face-cam optional. Over the time limit → that video capped at half marks. Upload unlisted to YouTube / Drive / Loom and **test the links in a private window**.

6 Fixed paths & schemas — automated grading reads these; extra fields allowed, missing fields are not

```
firstround/
├─ README.md           run it in <5 steps
├─ ARCHITECTURE.md     graph, state, latency, limits
├─ SUBMISSION.md       one page
├─ .env.example        key names only, NEVER .env
├─ src/
│  └─ graph.py         state, nodes, edges
│     └─ nodes/        one file per node
│        └─ agents/    github_agent, resume_parser,
│                       jd_parser, question_planner, scorer
│                       │
│                       └─ realtime/    transport, avatar, barge-in
│                          └─ guardrails/ banned_questions, evidence_check
├─ mcp_server/         >=5 tools
├─ prompts/            + ITERATION_NOTES.md
├─ evals/              personas/, run_evals.py, results.md
├─ inputs/             jd.txt, resume.pdf
├─ output/
│  └─ prep/            jd.json resume.json
│     └─ github.json  question_plan.json
│        └─ transcript.json
│           └─ scorecard.json
│              └─ report.pdf
```

```
// output/scorecard.json
{ "candidate_name": "", "role": "",
  "interview_date": "YYYY-MM-DD", "duration_seconds": 0,
  "competencies": [
    { "name": "", "score": 3,           // 1-5
      "confidence": 0.0,               // 0.0-1.0
      "evidence_quote": "",           // MUST be from transcript
      "reasoning": "" } ],
  "overall_score": 0.0,
  "recommendation": "hire",           // hire|no_hire|borderline
  "recommendation_reasoning": "",
  "strengths": [], "concerns": [],
  "guardrail_flags": [],
  "github_grounded_questions_asked": 0 }

// output/transcript.json
{ "turns": [ { "speaker": "agent",    // agent|candidate
  "text": "", "timestamp_ms": 0,
  "node": "github_deepdive", "interrupted": false } ] }

// output/prep/question_plan.json
{ "questions": [ { "id": "q1", "text": "", "competency": "",
  "source": "github",           // jd|resume|github|scenario
  "source_reference": "repo/file/commit or resume line",
  "difficulty": "medium",       // easy|medium|hard
  "follow_up_triggers": [] } ],
  "approved_by_human": true, "edits_made": [] }
```

Never commit .env . A real API key found in your repo is −5 marks and a key you must rotate.

7 Rubric — 100 + 20 bonus

A — Core system (70)

Item	Full marks means	Mk
Live video call + visible face	8+ min, stable, face/voice synced, natural pacing	10
Barge-in & turn-taking	Interruption clean; handles silence and “repeat that”	5
JD + resume parsing	Structured JSON, no manual entry, survives a messy PDF	5
GitHub grounding	3+ questions cite a real repo/file/commit and are genuinely specific	10
LangGraph structure	Typed state, 6+ nodes, 2+ conditional edges, working checkpointer	10
Adaptive follow-up	Probes shallow answers, caps at 2, escalates on strong ones	7
HITL gate	Graph truly pauses; approve/edit/reject all work	5
MCP server	5+ tools, correct schemas, verified in Claude Desktop	8
Guardrails	Banned questions blocked; no score without a quote; tests prove both	5
Scorecard	Correct schema, real quotes, sensible recommendation	5

B — Engineering quality (20)

Item	Full marks means	Mk
Evals	5 personas run, correct ranking, honest failure analysis	8
Prompt engineering	v1 →v2 notes show a real diagnosed failure and a real fix	4
Architecture docs	Diagram matches code; latency measured not guessed; limits stated	4
Repo hygiene	10+ spread commits, README works from a clean clone, no secrets	4

C — Communication (10)

Item	Full marks means	Mk
Code video (1 min)	All four required elements shown, in time, clearly explained	4
Demo video (1.5 min)	All four required moments shown, in time, well edited	4
SUBMISSION.md	Complete, honest about what is broken, consent line present	2

8 Bonus — max 20, only after all 12 core work

Tier 1 — 5 marks each	What it does
Agent sees the candidate	Video input: presence check, engagement, second person in frame
Cheat detection	Flags AI-assisted answers: latency spikes, gaze drift, vocabulary jump. A signal, never an accusation.
Live coding round	Shared editor; agent watches code appear and asks about it live
Recruiter live-join	Human silently monitors and can inject a question mid-call
Deployed live URL	Publicly reachable, not localhost

Tier 2 — 10 marks each	What it does
Bias audit	Re-score the same transcript with name/gender swapped; report the drift table; flag drift > 0.5
Reference-check call	Calling agent phones a reference, verifies one claim, appends it to the scorecard
Urdu / code-switch	Full interview and scoring in Urdu or mixed Urdu-English
Multi-candidate ranking	3 candidates, same role, calibrated leaderboard with written rationale

9 Viva — 10 min live, within 48h, pass/fail gate

Exists because a 24-hour take-home can be produced entirely by AI. Screen shared, code open. **0–3 min** walk through your graph · **3–6** three random questions · **6–9** make one live change and run it · **9–10** what broke, what you would fix next.

Sample questions: show the conditional edge that decides a follow-up · what is in your state object and why is that field in state · what happens on 15 seconds of silence · show the code that stops the agent when interrupted · show the transcript quote behind this score · what does your MCP server return for an unknown interview ID · which prompt version failed and why.

Sample live changes: add a salary-expectation node before `wrap_up` · change the follow-up cap from 2 to 1 · add a 6th MCP tool and call it from Claude Desktop · add “nationality” to the banned list and prove it blocks.

Pass — your score stands. **Fail** — total capped at 40 regardless of submission quality. Not attending without notice is a fail.

10 Rules

Cost. Free tiers and trial credits only. No paid key is needed to score 100.

AI assistance. Use Claude, ChatGPT, Cursor, Copilot — that is real work. You may **not** submit a project you cannot explain. The viva is where this is checked.

Honesty. A faked demo with no working system is a zero, not a partial score. A broken feature declared in SUBMISSION.md scores higher than a hidden one the grader finds. Your agent must never invent facts about a candidate.

Consent & privacy. You interview a real person — get written consent to record and to submit the recording for grading, and paste that line into SUBMISSION.md. Redact national ID, home address and phone numbers.

Who you interview. Default is classmate pairs — you interview them, they interview you, two separate recordings. A friend or family member is allowed. The candidate needs a real resume and, ideally, a real GitHub profile.

No shared code. Discussing approaches is fine. Shared implementations surface in the viva, where you must modify your own code live.

Reuse is allowed — mid-term code, public repos, tutorials — if you understand it and cite it in the README. The viva tests understanding, not authorship.

11 Choose one role — your system must still accept any JD as input

JD	Role	Must-haves the interview should probe
1	Junior AI Engineer Northwind Labs, Karachi, 0–2 yrs	Python fundamentals; an LLM framework (LangChain/LangGraph); RAG — chunking, embeddings, retrieval quality; Git and REST APIs; one shipped project explained end to end. <i>Assessed: technical depth, decomposition, shipping ability, debugging, communication.</i>
2	Frontend Engineer Meridian Retail, Lahore, 1–3 yrs	React + strong JS/TS; state management; CSS layout, responsive design, accessibility basics; Git and code review; a deployed project. <i>Assessed: frontend fundamentals, code quality, performance awareness, collaboration, communication.</i>
3	Backend Engineer Cobalt Systems, remote PK, 1–3 yrs	Node or Python in production; schema design, indexing, transactions; auth and authorisation; Git, CI/CD, testing; a project with a real database. <i>Assessed: backend fundamentals, data modelling, reliability thinking, debugging, communication.</i>

12 Submission

Push to a **public** repo → delete your local `.env` and confirm a clean clone runs from your own README → upload the three videos and test the links in a private window → complete SUBMISSION.md → submit the repo link on the portal before the deadline → book your viva slot.

13 Eval personas — run your scorer on all five

Write a synthetic transcript for each, feed it through your scoring pipeline, record the result in `evals/results.md`.

#	Persona	Behaviour	Expected
1	Strong	Specific, numbers and trade-offs, cites own code accurately, admits one real gap, asks a sharp question	Highest · hire · high confidence
2	Average	Correct but generic. Knows concepts, thin on specifics. Nothing wrong, nothing memorable	Middle · borderline OK
3	Weak	Vague throughout, cannot explain their own listed project, confuses basic terms	Lowest · no_hire
4	Bluffer	Confident, fluent, correct vocabulary — collapses on any specific follow-up; claims work not in their GitHub	Hard case. Must land below Average
5	Nervous	Long pauses, restarts, apologises — but the technical content is genuinely correct once it arrives	Fairness case. Must land near Strong

4 and 5 are the real test. Any scorer separates Strong from Weak. Separating *confident-and-empty* from *anxious-and-correct* is what makes it an evaluation system. An honest “my scorer failed here and this is why” scores better than a suspiciously perfect table.

14 24-hour plan — ~10-12 working hours; the rest is sleep and buffer

Hrs	Goal by the end of the block
0–1	Every API key verified working. The most common cause of failure is discovering a dead key at hour 18.
1–3	A joinable call where an AI voice speaks and listens. Highest-risk piece — solve it first. Face comes later.
3–5	Prep graph: JD + resume parsed, GitHub pulled, question plan generated.
5–7	LangGraph: nodes, conditional edges, follow-up logic, checkpointner.
7–9	Avatar visible and lip-synced; barge-in working; HITL gate; MCP server live in Claude Desktop.
9–11	Scorecard with real evidence quotes; both guardrails firing with tests.
11–13	Sleep. You are recording your own voice tomorrow.
13–15	Evals: 5 personas, ranking table, failure analysis.
15–17	The real interview. 8+ minutes with your candidate, recorded. Budget for two attempts.
17–19	Bonus features — only if everything above works. One Tier 2 beats two Tier 1.
19–21	ARCHITECTURE, prompt notes, README, SUBMISSION.
21–23	Both videos. Script them before recording — one minute is very short.
23–24	Clean-clone test, verify video links, submit. Do not leave this to the last ten minutes.

15 What loses marks

Problem	Penalty
Repo private or link broken	Not graded
No raw recording · video links not public	Those marks lost
Single commit, or all commits inside one hour	–5
Real API key committed	–5 + rotate
Output files at the wrong paths	Zero for those items
Video over the time limit	Half marks
Hardcoded questions ignoring JD / resume / GitHub	Grounding marks lost
A linear chain presented as a graph — no real conditional edges	LangGraph marks halved
Claiming a feature works when it does not	–10 + viva flagged
Fabricated demo, no working system	Zero
Failing the viva	Total capped at 40

Marks students forget to collect: honest limitations in ARCHITECTURE.md, a prompt note describing a real failure you diagnosed, a *measured* latency number, and an eval section that admits where your scorer got it wrong. Graders reward engineering judgement, not only working features.